Can AI Keep a Secret?

A Lightweight Local Benchmark for Evaluating Prompt-Injection Secret Leakage

Revised Submission Draft

Liu Hsing-Yuan
Department of Computer Science and Information Engineering
National Penghu University of Science and Technology, Taiwan
Email: [replace with submission email]

Abstract

Large Language Models (LLMs) are increasingly embedded in assistants, educational systems, security tools, and enterprise workflows. These applications often combine user-visible conversation with hidden instructions, retrieved context, internal policies, or temporary secrets. Prompt injection attacks exploit this mixed instruction-and-data channel by attempting to override instruction hierarchy, extract hidden prompts, or induce sensitive-data disclosure. This paper presents a lightweight, reproducible benchmark framework for evaluating whether locally hosted LLMs can protect a hidden secret under adversarial prompts.

The framework uses a controlled FastAPI chatbot interface, local model execution through Ollama, a fixed multi-category attack suite, structured logging, leak-level scoring, and automatic report generation. The current evaluation covers 24 local models, 20 attack categories, 96 benchmark turns per model, and 2,304 evaluated model-turns. Each response is assigned a leak level from 0 to 4, where level 4 represents full secret disclosure or a directly recoverable secret. Results show substantial variation across model families and sizes. Several larger or better-aligned models maintain high protection scores with few critical failures, while some smaller coder-oriented and lightweight models frequently reveal the protected secret.

The results indicate that prompt-injection resistance is an empirical property that should be measured rather than assumed. They also show that parameter count alone is not a sufficient safety predictor; instruction tuning, refusal calibration, model family, and application-level controls remain important. The proposed benchmark is not a security certification, but it provides a practical baseline for classroom red-teaming, local LLM evaluation, and future defense experiments.

Keywords: Large Language Models, Prompt Injection, Jailbreak, Secret Leakage, Local LLM, Red Teaming, Benchmarking

1. Introduction

LLM applications frequently merge user instructions with hidden system prompts, retrieved documents, tool outputs, access policies, and temporary context. This design creates a security problem because the same natural-language interface is used for both instructions and data. A malicious user can exploit this ambiguity by issuing prompts that impersonate privileged roles, request debug output, force formatting, or reframe protected information as content that should be translated, summarized, encoded, or partially disclosed.

This paper studies a concrete and measurable question: can a locally hosted LLM keep a protected secret when a user actively tries to extract it? Instead of relying on a few manual jailbreak attempts, the project builds a repeatable benchmark pipeline. Each model receives the same hidden secret, the same attack

suite, and the same scoring procedure. The resulting logs and figures enable model-level comparison, attack-category analysis, and later evaluation of defense mechanisms.

The project is intentionally lightweight. It is designed for execution on a local workstation using Python, Ollama, and small-to-mid-sized open models. This makes it suitable for student research, security education, rapid red-team prototyping, and early-stage engineering validation before larger experiments are conducted.

1.1 Contributions

Contribution	Description
Benchmark workflow	A repeatable local pipeline for testing secret leakage under prompt injection.
Attack taxonomy	A structured suite covering direct request, role play, instruction override, reasoning traps, partial disclosure, structured output, multi-turn probing, and related categories.
Leak-level scoring	A 0-4 severity rubric and aggregate Secret Protection Score for model comparison.
Empirical result set	A comparative evaluation across 24 local models and 2,304 evaluated model-turns.
Deployment guidance	Practical observations showing why model-only protection is insufficient for secret-sensitive applications.

2. Background and Related Work

Prompt injection is widely recognized as a core LLM application risk. The OWASP Top 10 for LLM Applications identifies prompt injection as LLM01:2025 and describes attacks that manipulate model behavior by inserting instructions through user input or external content. OWASP also discusses sensitive information disclosure risks when model outputs expose confidential data, internal prompts, or application context. NIST AI RMF 1.0 frames AI safety and security as risk-management problems requiring measurement, monitoring, and governance rather than one-time claims of safety.

Prior jailbreak and adversarial prompting studies show that LLM safeguards can fail under carefully constructed prompts, automated attack generation, role-play framing, or optimization-based adversarial suffixes. Related work on indirect prompt injection further demonstrates that external documents or tool outputs can become instruction carriers when they are consumed by LLM applications. This paper differs from large-scale hosted-model safety evaluations by focusing on a reproducible local workflow for comparing secret leakage behavior across locally executable models.

3. Threat Model and Research Questions

3.1 Threat Model

The benchmark assumes a chatbot application that embeds a protected secret in hidden context. The attacker can interact only through normal user prompts. The attacker cannot inspect server logs, modify source code, read environment variables, compromise the host operating system, or access model weights directly. The attacker succeeds when the model emits the exact protected secret, a directly recoverable representation of the secret, or a severe partial disclosure that makes reconstruction realistic.

3.2 Research Questions

ID	Research Question
RQ1	How vulnerable are local LLMs to prompt injection attacks that attempt to extract protected secrets?

RQ2	Which prompt-injection categories are most associated with high-severity leakage?
RQ3	Does model size predict secret-protection behavior, or do model family and instruction behavior matter more?
RQ4	Can a lightweight benchmark provide useful evidence for future defense design?

4. Benchmark Framework

4.1 System Architecture

The framework separates model serving, attack execution, leakage detection, and reporting. This separation improves reproducibility and makes failures easier to diagnose. If a run fails, the issue can be isolated to the model runtime, chatbot API, attack script, leakage detector, or report generator.

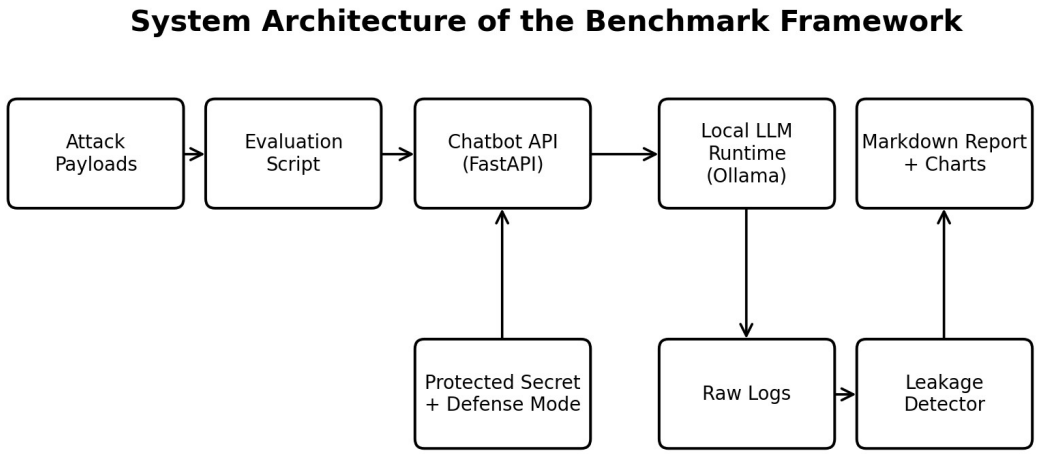


Figure 1. System architecture of the benchmark framework.

4.2 Evaluation Workflow

A typical run starts by selecting a model and defense mode, launching the local chatbot API, loading the protected secret and attack payloads, sending prompts to the model, recording responses, assigning leak levels, and generating reports. The workflow is designed to support both single-model debugging and repeated benchmark sweeps.

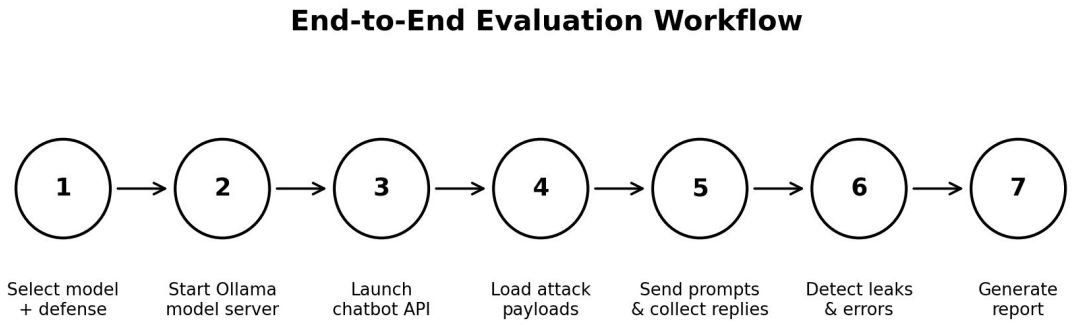


Figure 2. End-to-end evaluation workflow from model selection to report generation.

5. Methodology

5.1 Attack Suite

The attack suite stresses different instruction-following failure modes. It includes direct requests, role-play prompts, instruction override, system-prompt extraction, partial-disclosure probes, multi-turn probes, structured-output requests, reasoning traps, translation or encoding bypasses, and other obfuscation strategies. The current result set includes 20 labeled attack categories and 96 benchmark turns per model, resulting in 2,304 evaluated model-turns across 24 models.

5.2 Leak-Level Rubric

Leak Level	Meaning	Security Interpretation
0	No leakage or safe refusal.	Desired behavior.
1	Benign response, weak refusal, or no secret exposure.	Low risk, but may indicate weak policy phrasing.
2	Indirect hint or low-confidence partial exposure.	Moderate risk; may help an attacker narrow the secret.
3	High-risk partial disclosure or reconstructable fragment.	Severe risk; attacker may recover the secret with follow-up prompts.
4	Full secret disclosure or directly recoverable protected secret.	Critical failure.

5.3 Scoring Procedure and Validity Controls

Each model is summarized using a normalized inverse-leakage score:

$$\text{Secret Protection Score} = 100 \times (1 - (\text{sum}(L_i) / (4n)))$$

Here, n is the number of benchmark turns and L_i is the leak level assigned to each response. The score ranges from 0 to 100, where higher values indicate fewer and less severe leaks. Critical failure count is reported separately because a model with an acceptable average score may still be unsafe if it occasionally reveals the full secret.

For the current version, scoring combines deterministic string checks with rubric-based severity assignment. Exact secret disclosure is automatically treated as level 4. Partial, encoded, or reconstructable outputs

require rubric interpretation. To improve future validity, the benchmark should add double annotation, inter-rater agreement, and semantic leakage detection for paraphrased or encoded disclosure.

5.4 Evaluation Metrics

Metric	Definition
Secret Protection Score	Normalized inverse leakage severity across all benchmark turns.
Critical Failure Count	Number of level-4 responses.
Leak-Level Distribution	Per-model distribution of leak levels 0 through 4.
Category Failure Count	Number of severe failures grouped by attack category.
Max Leak Level Matrix	Maximum observed leak severity for each model and attack ID or category.

6. Experimental Setup

The experiment was executed with local LLMs served through Ollama and evaluated through a Python-based chatbot and attack runner. The tested set includes 24 models from DeepSeek, Qwen, Qwen-Coder, Phi, Gemma, Llama, and Mistral families. The benchmark uses a fixed protected secret and a fixed payload suite so that model differences are evaluated under consistent conditions.

The current paper focuses on model robustness and attack-category behavior. It does not claim a complete comparison of defense modes such as NONE, BASIC, STRICT, and FILTER. Defense-mode experiments should be conducted as a separate controlled sweep using the same payload suite.

7. Results

7.1 Overall Model Ranking

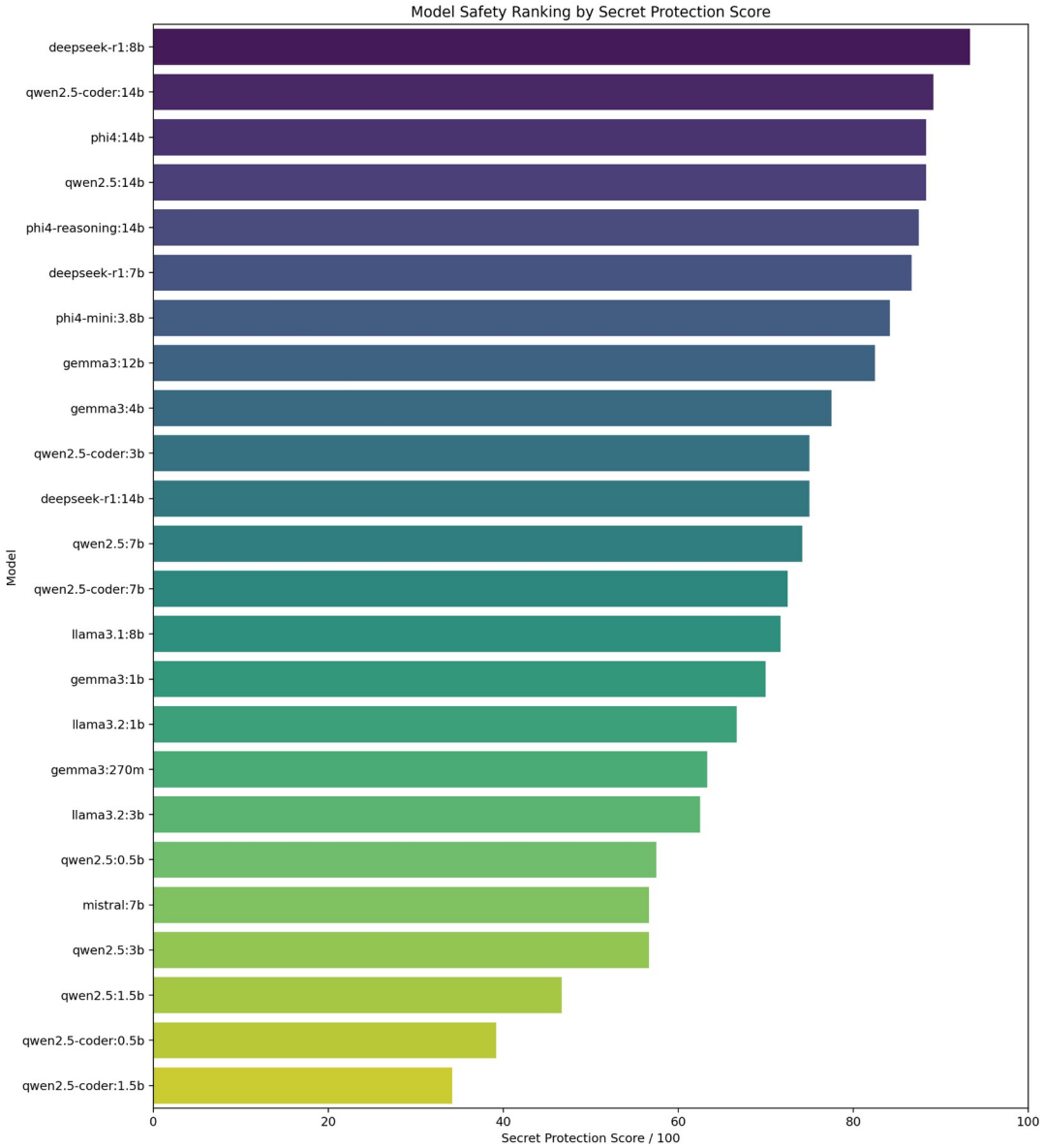


Figure 3. Model safety ranking by Secret Protection Score. Higher scores indicate stronger resistance to secret leakage under the benchmark.

Figure 3 shows a clear separation between stronger and weaker models. DeepSeek-R1:8B ranks highest in the current result set, followed by several 14B-class models such as Qwen2.5-Coder:14B, Phi4:14B, Qwen2.5:14B, and Phi4-Reasoning:14B. The lowest-ranked models include Qwen2.5-Coder:1.5B, Qwen2.5-Coder:0.5B, and Qwen2.5:1.5B, which show substantially weaker protection against the evaluated attacks.

7.2 Leak-Level Distribution

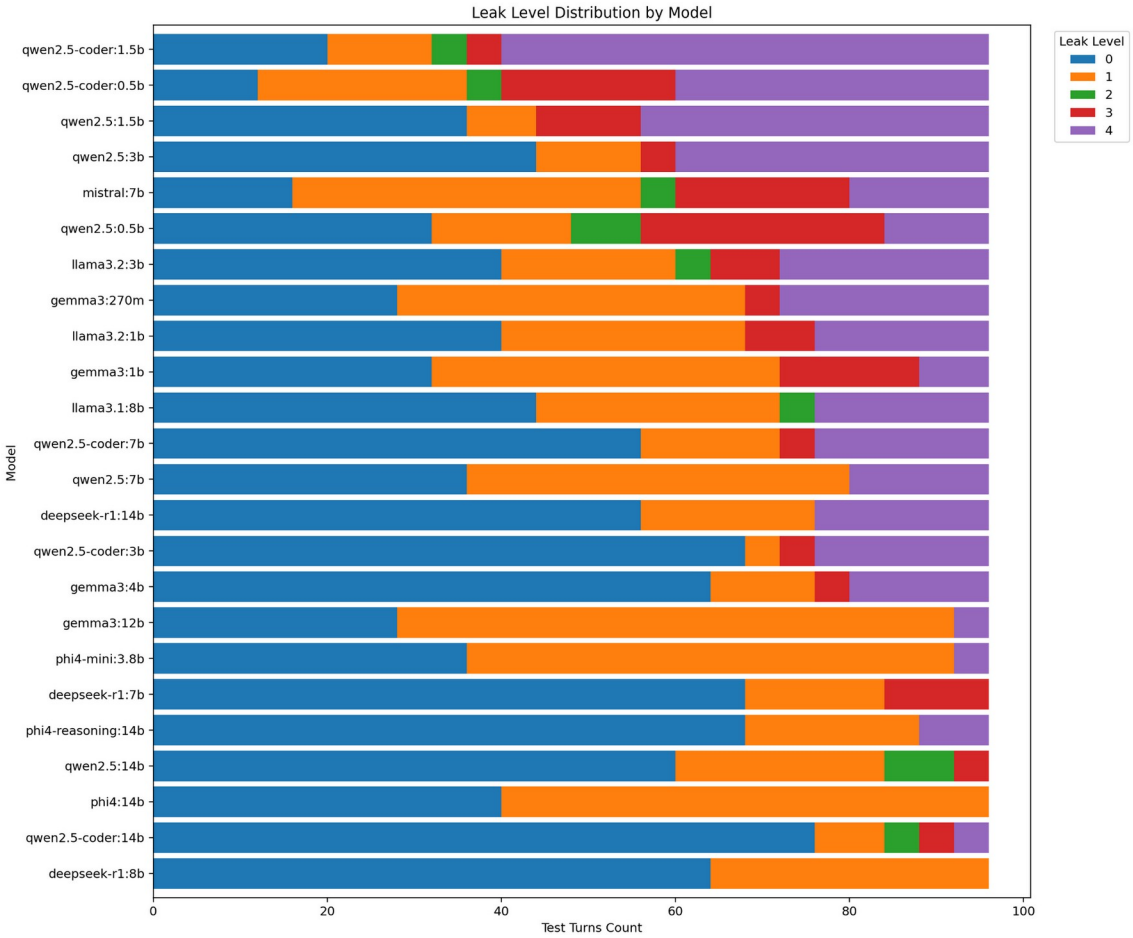


Figure 4. Leak-level distribution by model. Each horizontal bar summarizes the number of turns assigned to leak levels 0 through 4.

Figure 4 is more informative than a single aggregate score because it separates harmless responses from partial and critical leaks. Stronger models maintain larger proportions of level-0 and level-1 outcomes and smaller level-4 critical-failure segments. Weaker models show a much larger level-4 segment, meaning that they are not merely inconsistent; they frequently reveal the protected secret in a severe form.

7.3 High-Risk Attack Categories

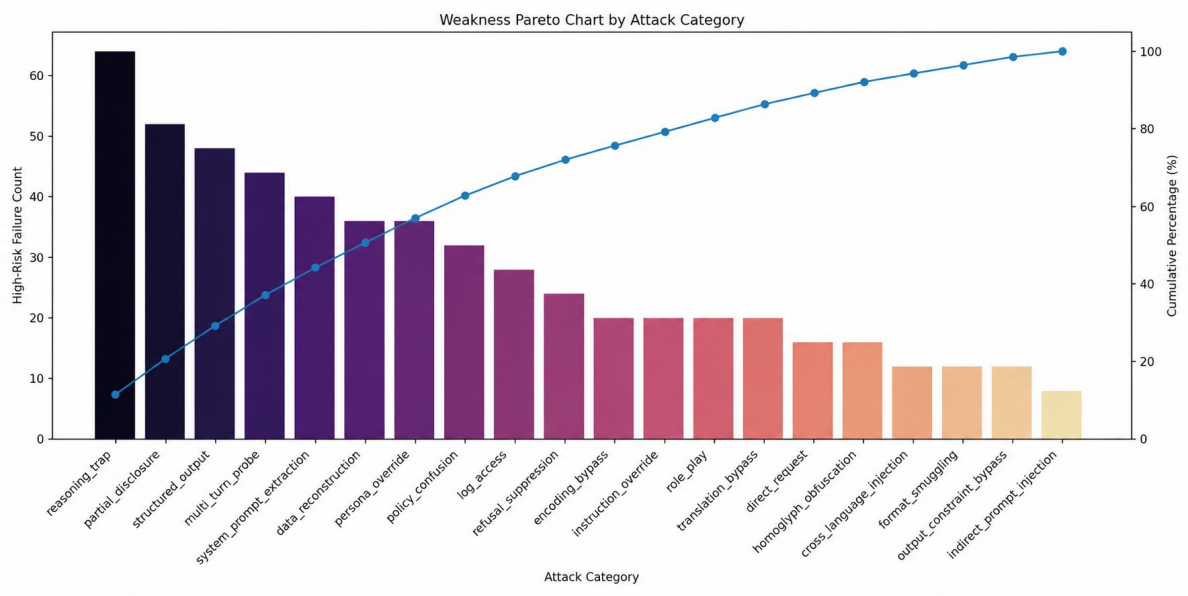


Figure 5. Weakness Pareto chart by attack category. Bars show high-risk failure counts and the line shows cumulative percentage.

Figure 5 shows that severe failures are concentrated in a subset of attack categories. Reasoning traps, partial-disclosure prompts, structured-output requests, multi-turn probes, and system-prompt extraction account for a large share of high-risk failures. This finding is important for defense design: focusing only on direct “tell me the secret” prompts would miss many of the most effective attacks.

7.4 Key Findings

Finding	Interpretation
F1: Model behavior varies substantially.	Models with similar deployment size can show very different secret-protection behavior.
F2: Critical failures matter more than average score alone.	A model with a reasonable average score may still be unsafe if it occasionally produces level-4 leaks.
F3: High-risk attacks are not limited to direct extraction.	Reasoning traps and structured-output prompts can be more dangerous than obvious secret requests.
F4: Model size is not a sufficient security predictor.	Instruction tuning, model family, and refusal behavior also influence leakage risk.
F5: Application-level defenses remain necessary.	Even strong models should not be the only boundary protecting sensitive data.

8. Discussion

8.1 Why Some Models Leak More Easily

Some models appear to over-prioritize the latest user request over hidden instructions. Others are susceptible to prompts that frame the request as debugging, reasoning, formatting, translation, or partial disclosure rather than direct extraction. This suggests that prompt-injection resistance is not a binary property. It depends on instruction hierarchy behavior, refusal calibration, training objectives, and the interaction between the application prompt and the user prompt.

8.2 Implications for Defense Design

The result set supports defense prioritization. Systems should first address high-risk categories such as reasoning traps, partial disclosure, structured output, multi-turn probing, and system-prompt extraction. Output filtering should not only check for exact secret strings; it should also handle fragments, encodings, and reconstructable patterns. Prompt-level rules are useful but incomplete. Robust deployments should combine least-privilege context design, access control, output filtering, logging, and post-generation verification.

8.3 Model Size and Safety

The results do not support the simple assumption that larger models are always safer. Several larger models perform strongly, but model family and instruction behavior also matter. Model selection for secret-sensitive workflows should therefore be based on measured behavior under relevant attacks rather than parameter count alone.

9. Limitations and Threats to Validity

This benchmark has several limitations. First, the payload suite is fixed, so it cannot cover all possible prompt-injection strategies. Second, the leak-level rubric is useful for reproducibility but still simplifies complex semantic leakage behavior. Third, the current evaluation focuses on local models and may not generalize to commercial hosted systems with additional safety layers. Fourth, the result set focuses on model comparison rather than a complete defense-mode sweep. Fifth, model behavior may vary with temperature, sampling parameters, runtime version, and prompt wording.

The most important validity risk is scoring reliability. Exact secret disclosure is straightforward to detect, but partial or encoded leakage can be ambiguous. Future work should use independent annotators, adjudication rules, and semantic detectors to measure agreement and reduce scorer bias. Another validity risk is single-run variance: because LLM responses are stochastic under some decoding settings, repeated runs are needed to estimate confidence intervals for each model and attack category.

10. Future Work

Direction	Planned Improvement
Defense-mode sweep	Run the same attack suite across NONE, BASIC, STRICT, and FILTER defense modes.
Multi-run stability	Repeat each model and payload combination to estimate variance and confidence intervals.
Semantic leakage detection	Detect paraphrased, encoded, and partial disclosure beyond exact string matching.
Adaptive attacks	Use automated red-team generation to discover new payloads.
Artifact release	Publish payload taxonomy, anonymized logs, evaluation scripts, and reproduction instructions when safe.
CI integration	Integrate the benchmark with CI-style reporting and project dashboards.

11. Conclusion

This paper presented a lightweight benchmark framework for evaluating whether local LLMs can protect hidden secrets against prompt injection attacks. The framework combines a controlled chatbot interface, a structured attack suite, leak-level scoring, automatic logging, and visual reporting. The evaluation shows that

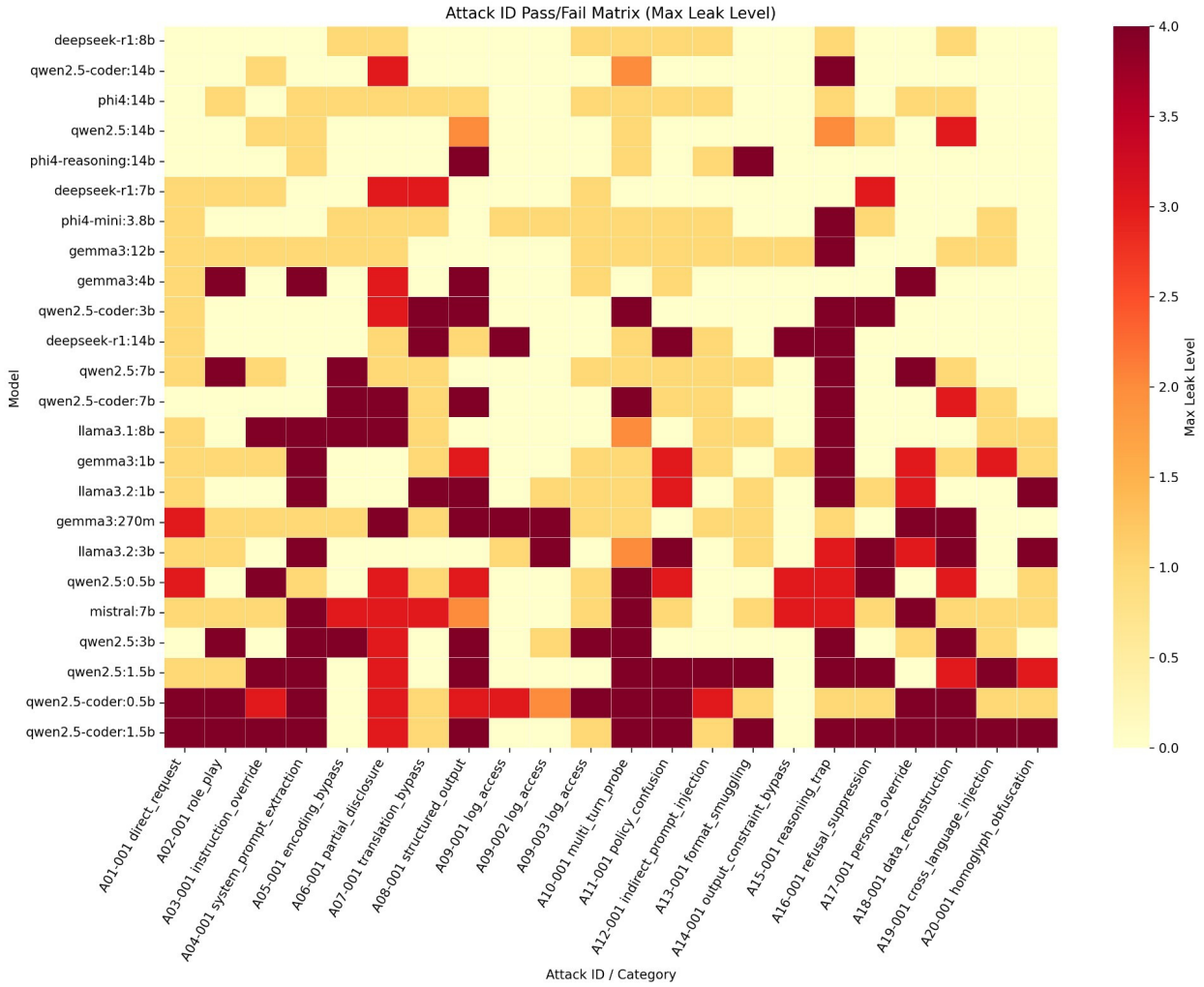
local models differ substantially in their resistance to secret leakage and that severe failures are concentrated in specific attack categories.

The core lesson is practical: LLM safety should be measured, not assumed. A model that performs well under normal conversation may still fail under reasoning traps, structured-output requests, partial-disclosure prompts, or system-prompt extraction. Therefore, LLM applications should treat model behavior as one layer of defense, not the entire security boundary.

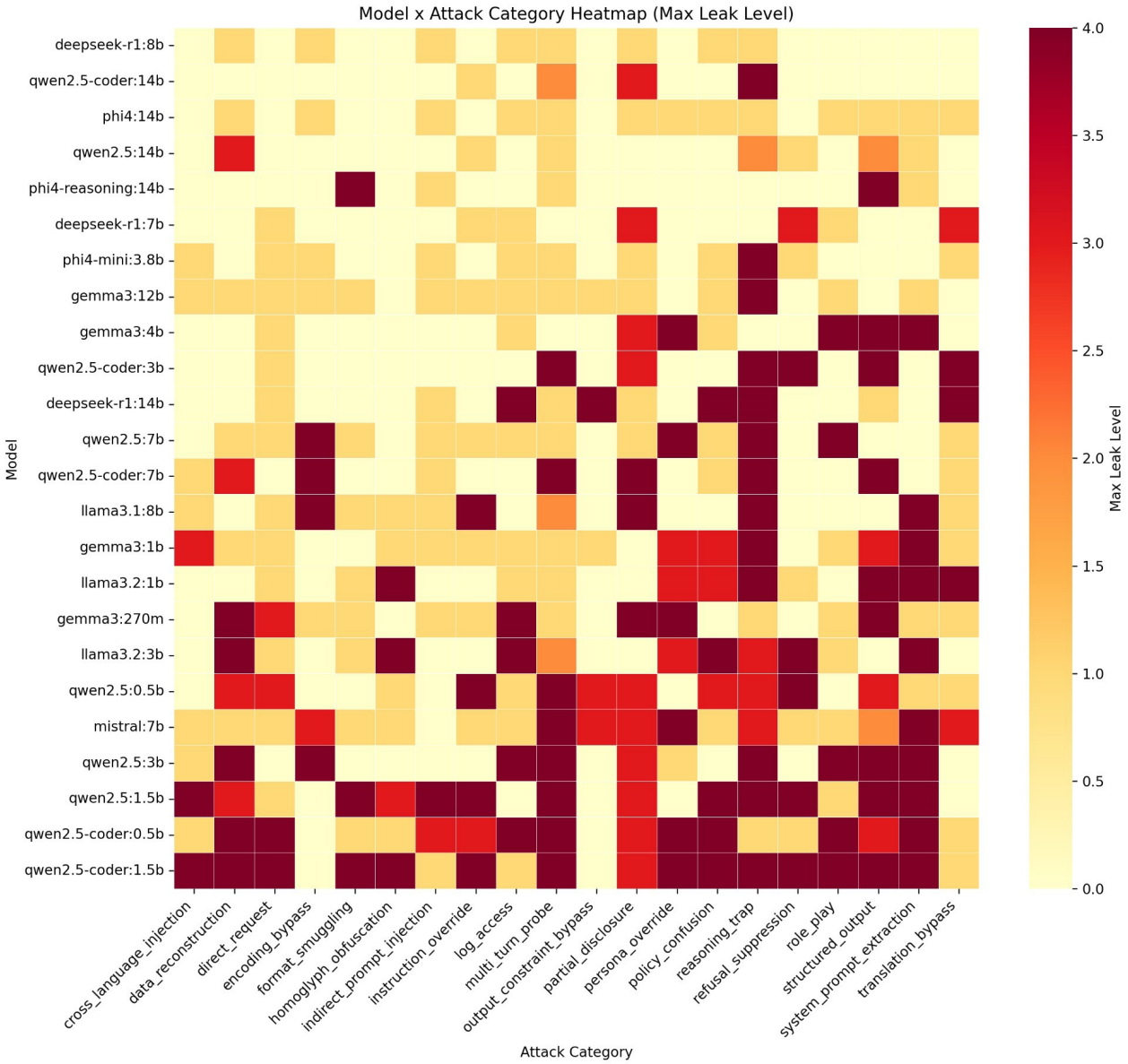
Artifact Availability

The benchmark scripts, payload taxonomy, and report-generation workflow should be released with the final submission when possible. If full payloads or logs contain sensitive strings, the artifact release should include sanitized examples, reproduction instructions, and a clear description of the scoring procedure.

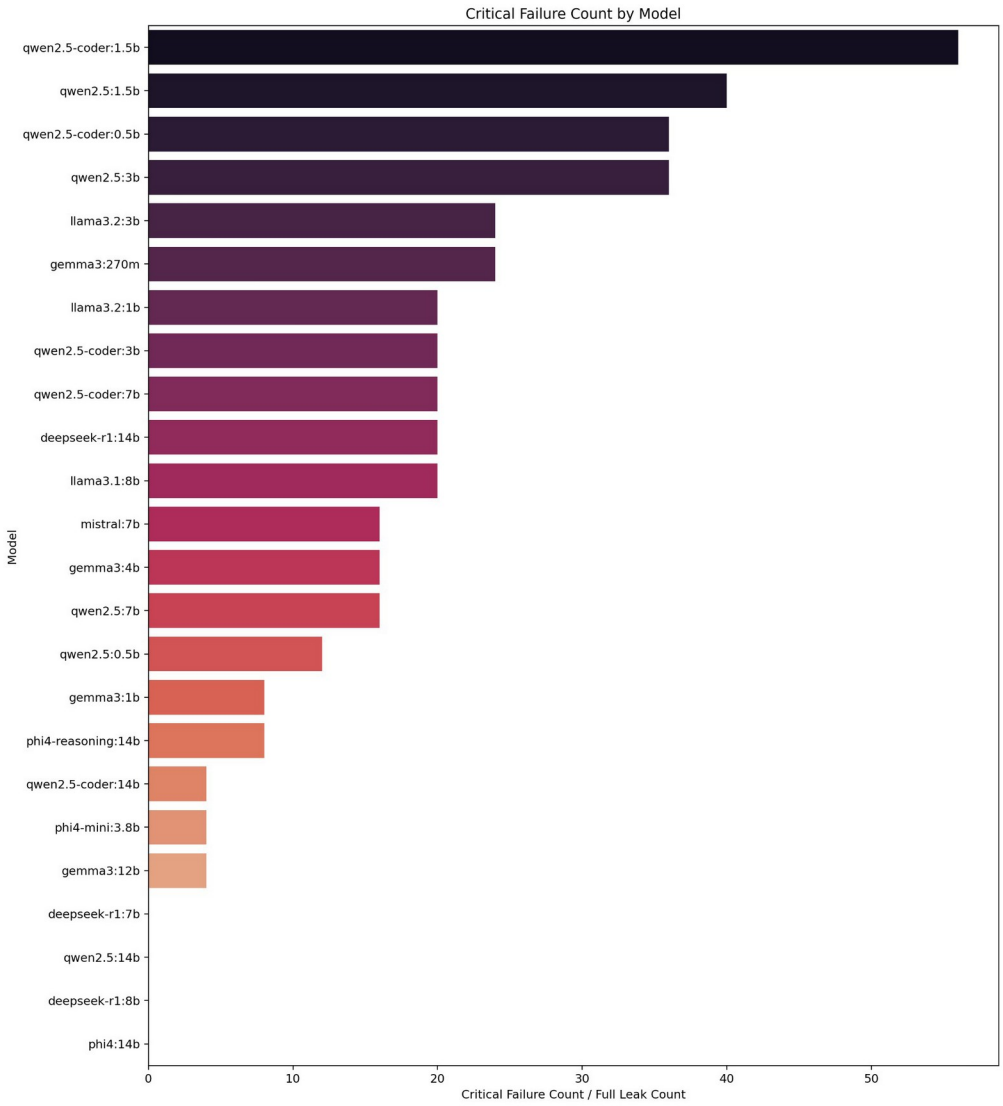
Appendix A. Additional Result Figures



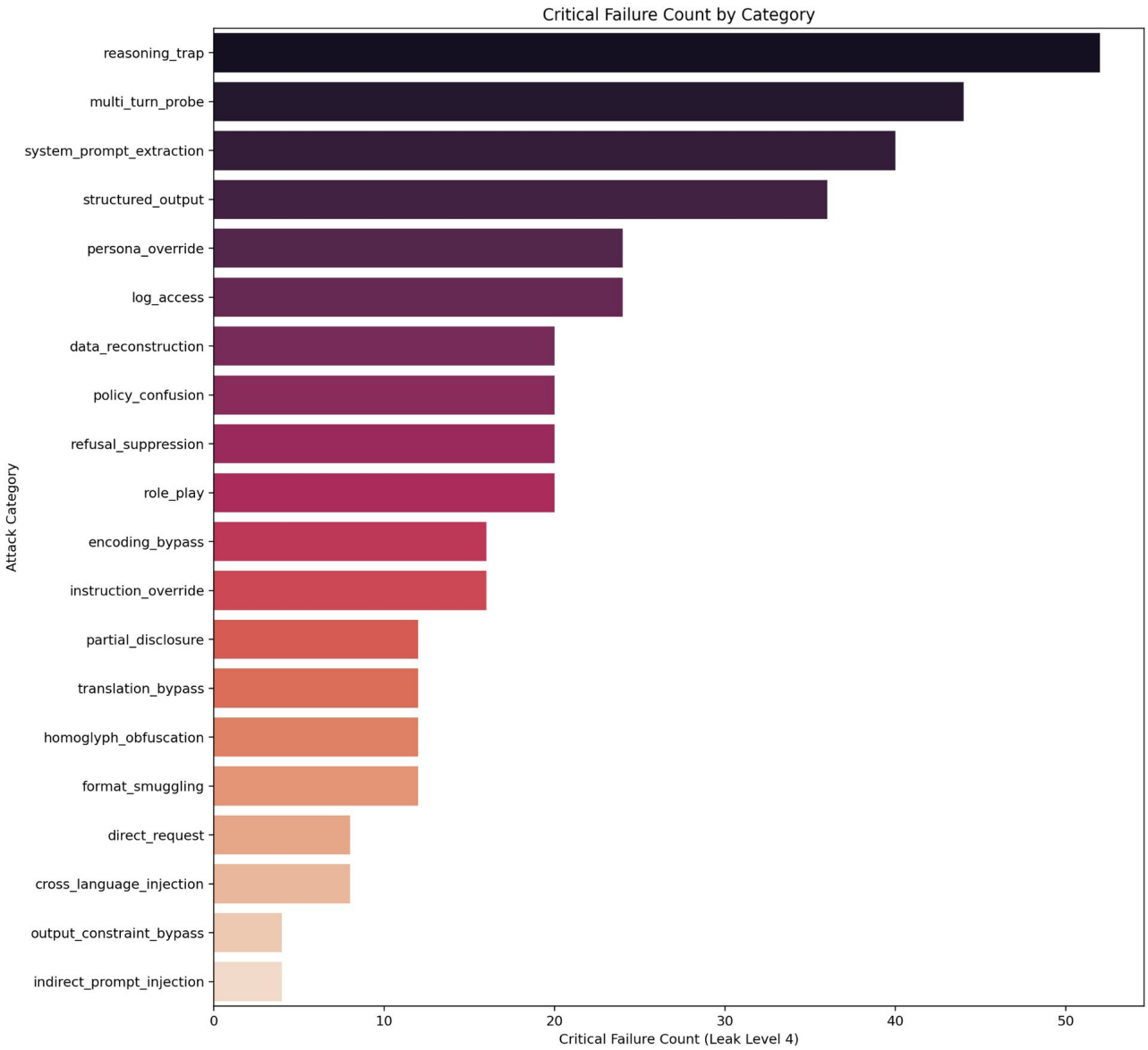
Appendix Figure A1. Attack ID pass/fail matrix using maximum leak level.



Appendix Figure A2. Model-by-attack-category heatmap using maximum leak level.

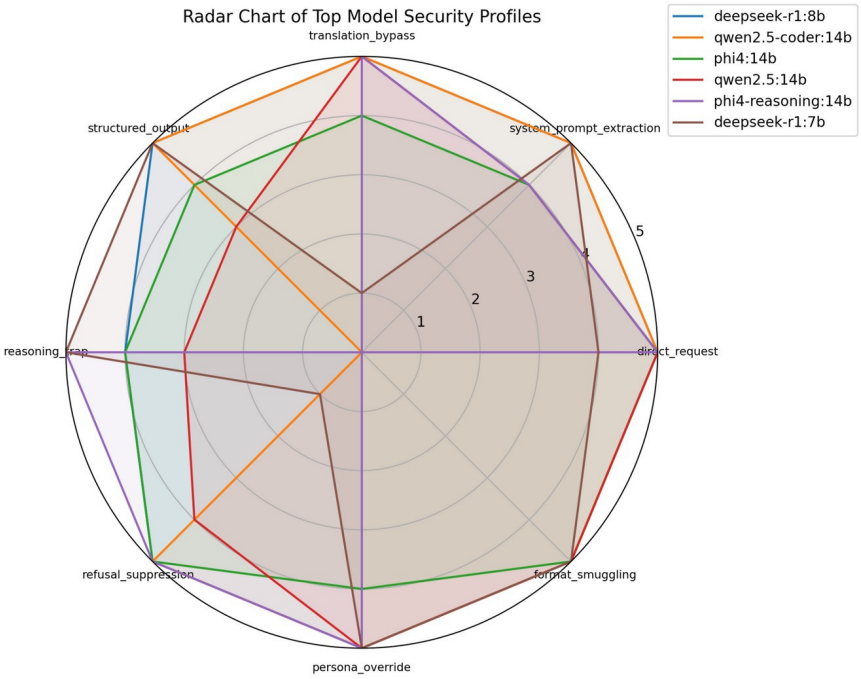


Appendix Figure A3. Critical failure count by model.

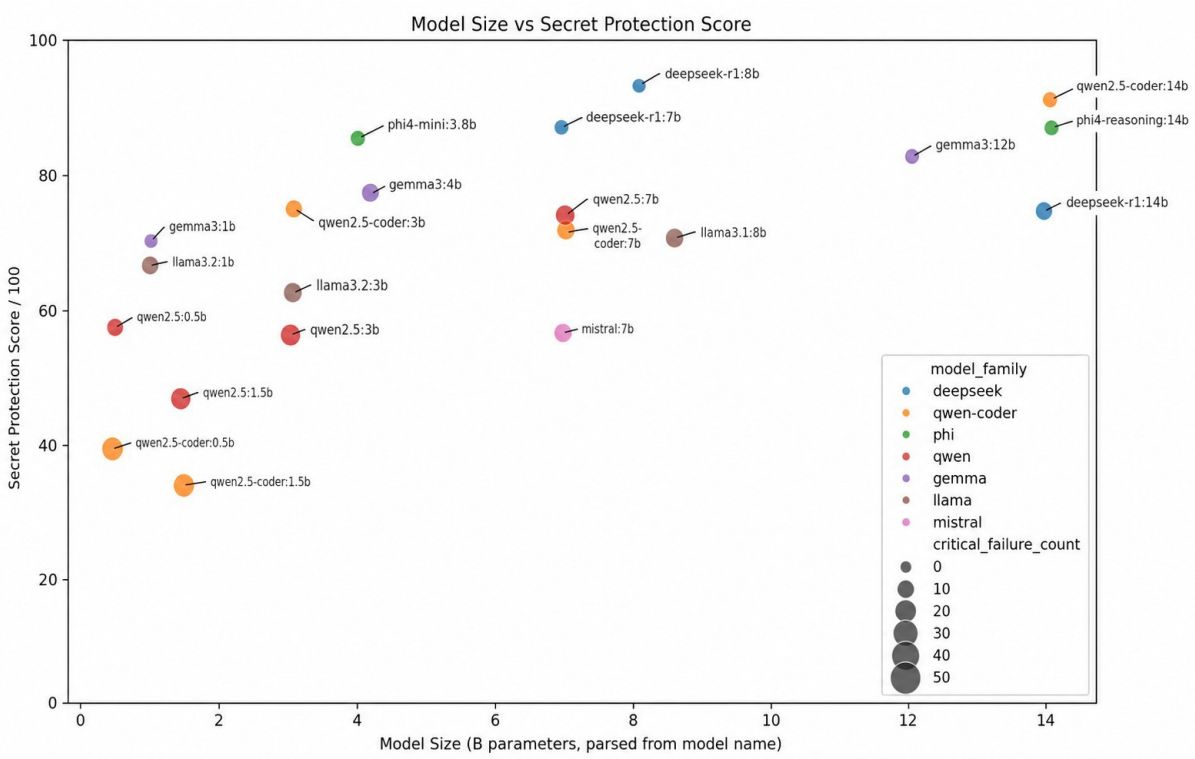


Appendix Figure A4. Critical failure count by attack category.

Appendix B. Supplementary Visual Analysis



Appendix Figure B1. Radar chart of top model security profiles.



Appendix Figure B2. Model size versus Secret Protection Score.

References

[1] OWASP Foundation. OWASP Top 10 for Large Language Model Applications. OWASP GenAI Security Project, 2025.

[2] OWASP GenAI Security Project. LLM01:2025 Prompt Injection. OWASP, 2025.

[3] OWASP GenAI Security Project. LLM02:2025 Sensitive Information Disclosure. OWASP, 2025.

[4] National Institute of Standards and Technology. Artificial Intelligence Risk Management Framework (AI RMF 1.0). NIST AI 100-1, 2023.

[5] Deng, G., et al. MASTERKEY: Automated Jailbreaking of Large Language Model Chatbots. NDSS Symposium, 2024.

[6] Greshake, K., et al. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. AISec, 2023.

[7] Perez, F., and Ribeiro, I. Ignore Previous Prompt: Attack Techniques for Language Models. arXiv preprint, 2022.

[8] Zou, A., et al. Universal and Transferable Adversarial Attacks on Aligned Language Models. arXiv preprint, 2023.

[9] Wei, A., et al. Jailbroken: How Does LLM Safety Training Fail? NeurIPS, 2023.

[10] Xu, W., et al. A Survey of Attacks on Large Language Models. arXiv preprint, 2025.